

Day 3: Introduction to Machine Learning with scikit-learn

Bruno Saraiva | ITQB NOVA

Nuno Martins | GIMM

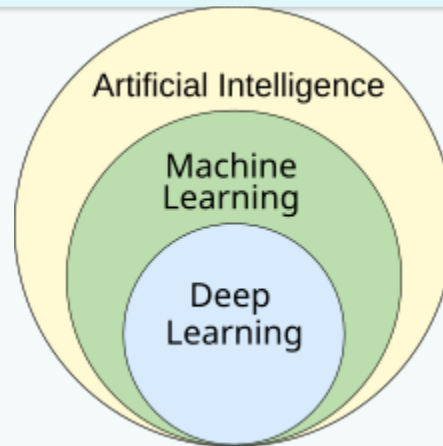
Gabriel Medeiros | GIMM

25 June 2026

What is machine learning?

Machine learning is a computational approach in which algorithms **learn patterns from data** and use those patterns to make predictions or decisions on new, unseen examples.

The rules are not programmed — they are learned from data.



Machine learning in practice



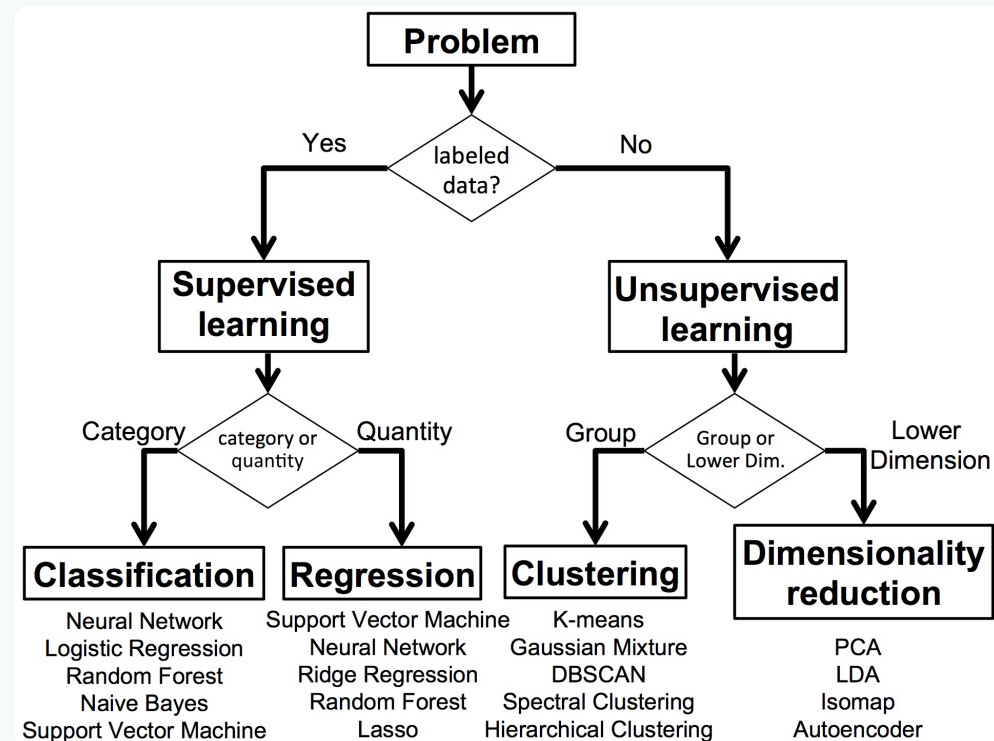
In practice, building a **machine learning model** is an **iterative process**:

1. Try a **model**
2. **Evaluate** what it has learned
3. Adjust the **data**, **features**, or model
4. **Repeat** until the results are useful

| Types of machine learning

Two broad families of algorithms:

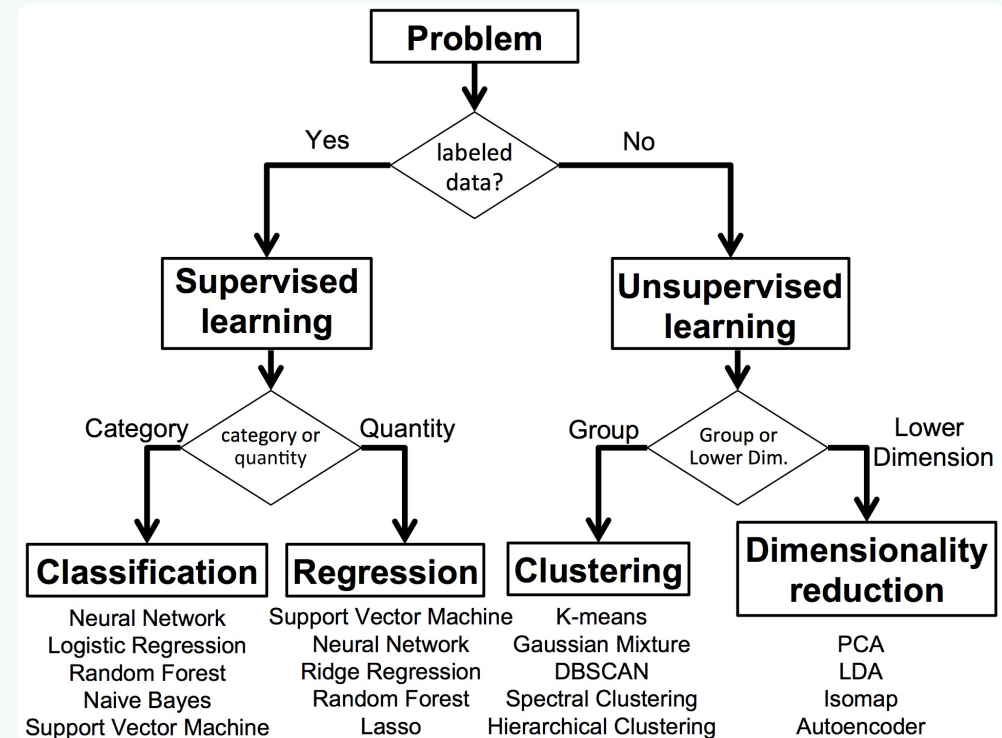
- **Supervised learning** — train with labeled examples
- **Unsupervised learning** — discover structure in unlabeled data



What we will cover today

In today's practical session we will work with:

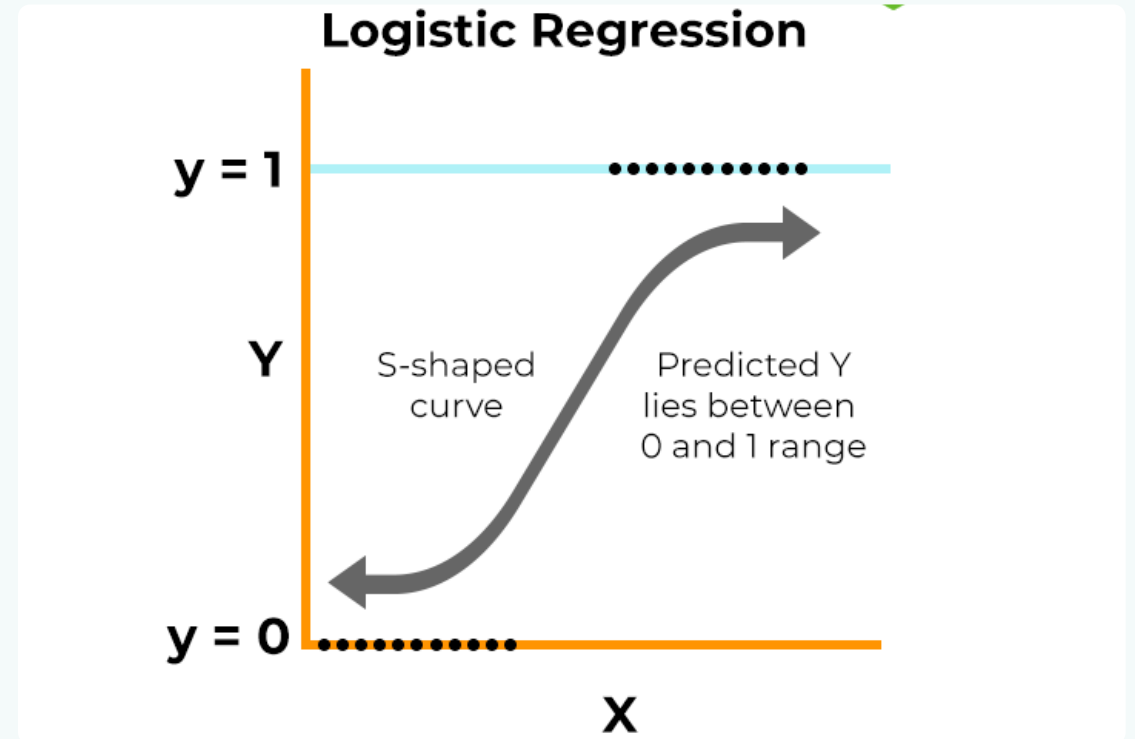
- **Classification** — assign a label to each sample
- **Clustering** — group samples by similarity
- **Segmentation quality** — measure how good a segmentation is



Supervised algorithms: logistic regression

Logistic regression is used for classification.

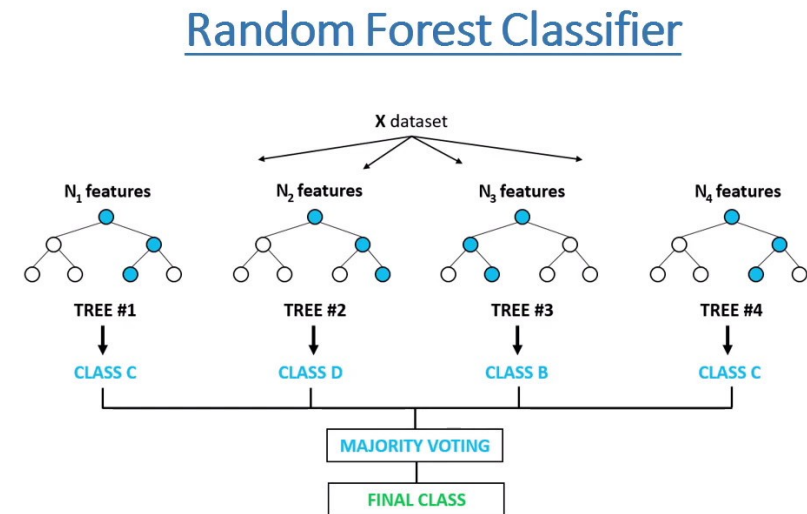
It estimates the **probability** that a sample belongs to a given **class** by fitting a **sigmoid** (S-shaped) curve to the **training data**.



Supervised algorithms: random forest

A **random forest** is an **ensemble** of **decision trees**.

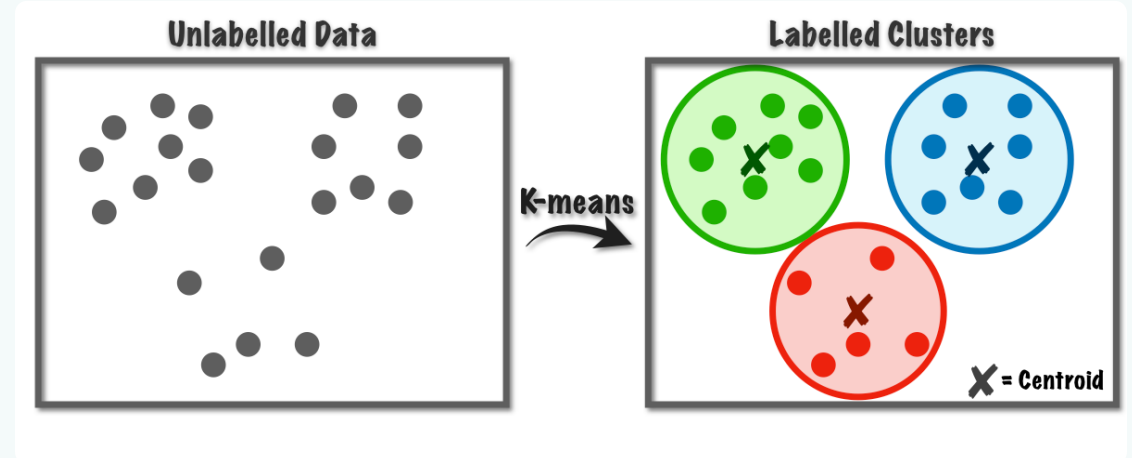
Each tree is trained on a random subset of the **data** and **features**. The final **prediction** is determined by combining the **votes** of all trees.



Unsupervised algorithms: k-means clustering

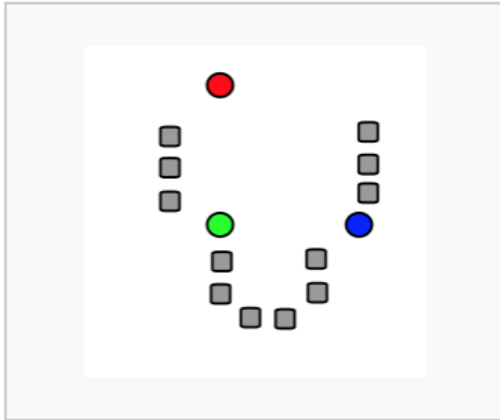
K-means groups data into a predefined number of clusters, **k**.

The algorithm assigns each point to the nearest **centroid**, then moves the centroids, and repeats until **assignments** no longer change.

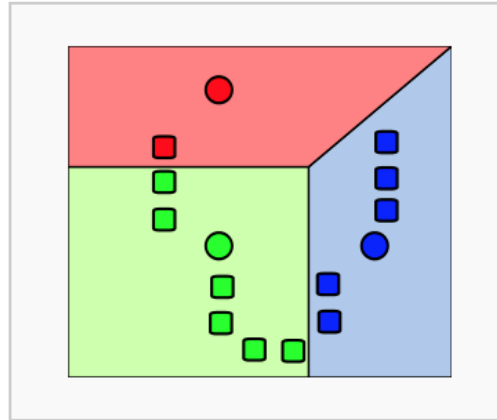


How does a model learn?

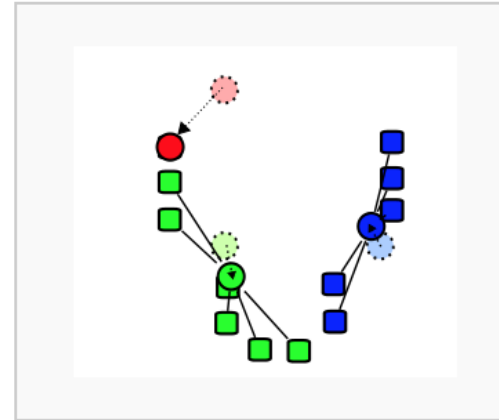
Demonstration of the standard algorithm



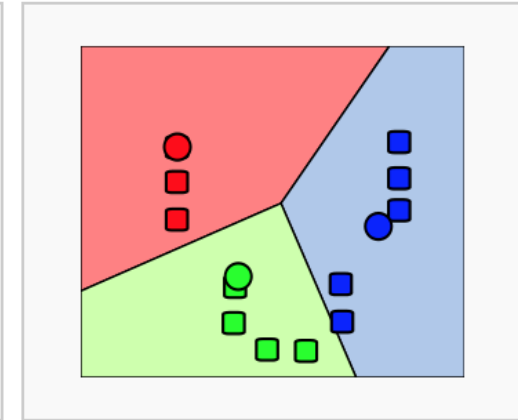
1. k initial "means" (in this case $k=3$) are randomly generated within the data domain (shown in color).



2. k clusters are created by associating every observation with the nearest mean. The partitions here represent the **Voronoi diagram** generated by the means.



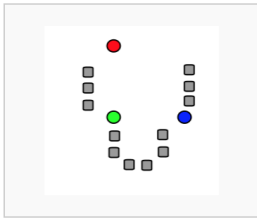
3. The **centroid** of each of the k clusters becomes the new mean.



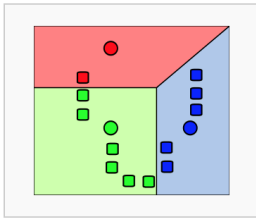
4. Steps 2 and 3 are repeated until convergence has been reached.

How does a trained model predict?

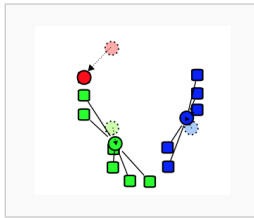
Demonstration of the standard algorithm



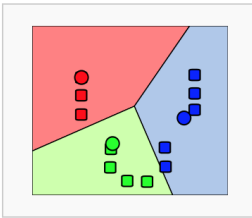
1. k initial "means" (in this case $k=3$) are randomly generated within the data domain (shown in color).



2. k clusters are created by associating every observation with the nearest mean. The partitions here represent the Voronoi diagram generated by the means.



3. The centroid of each of the k clusters becomes the new mean.



4. Steps 2 and 3 are repeated until convergence has been reached.

New samples are projected into the same **feature space** the model was trained on.

For **k-means**, a new point is assigned to the **cluster** whose **centroid** is closest:

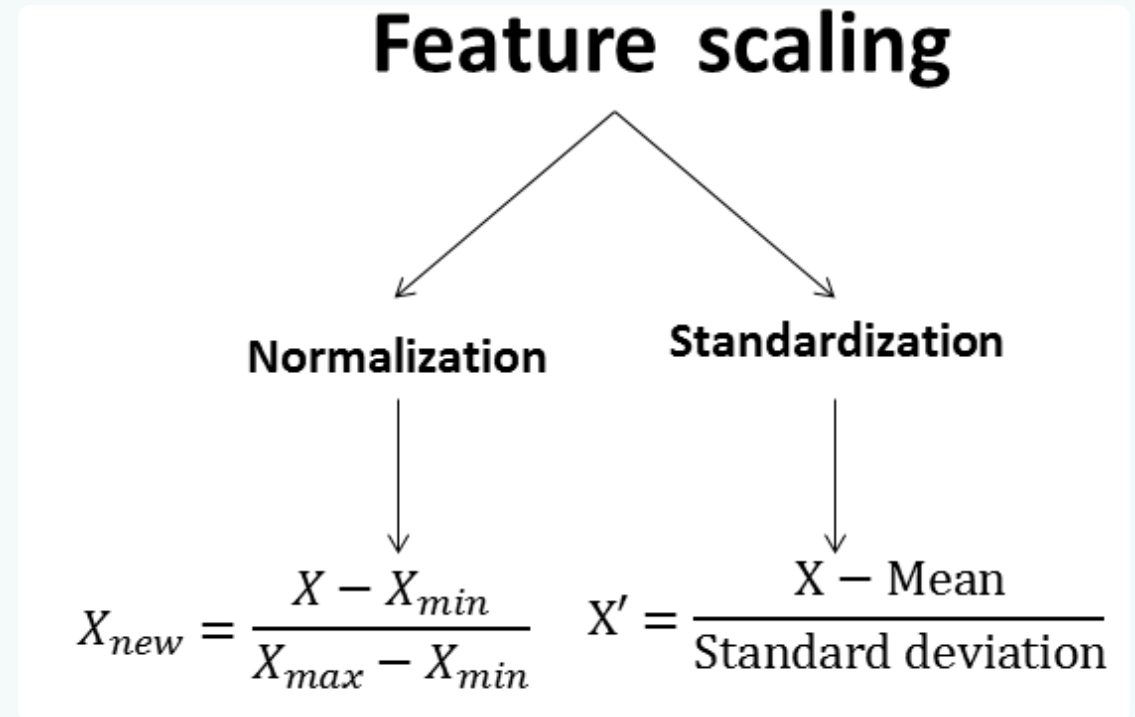
Closer to the blue centroid → assigned to the blue cluster.

Preparing data: feature scaling

Features measured on different **scales** can bias a model. **Feature scaling** ensures each feature contributes fairly.

Two common approaches:

- **Normalization:**
rescale values to the range [0, 1]
- **Standardization:**
rescale to mean = 0, std = 1



Preparing data: feature scaling

Before standardization

Area	Perimeter	Length	Width	Eccentricity	Irregularity
0.792537	1.690458	2.550826	-0.829119	2.268749	5.387692
1.645258	0.905877	0.670494	1.599536	-0.460539	-0.547352
-0.605365	-0.488935	-0.511774	0.022080	-0.700971	-0.096735
-0.102120	-0.140232	-0.386271	0.371875	-1.023854	-0.218632
0.163481	0.295647	0.135748	-0.185042	0.671941	0.181751
-0.381701	-0.401759	-0.256490	0.022080	-0.124583	-0.286012
1.477509	0.993052	0.670494	1.599536	-0.460539	-0.245740
-0.074162	-0.227408	-0.452346	0.261102	-0.999771	-0.415462
-1.304316	-0.488935	-0.564328	-1.077332	0.667521	1.254488
-0.689239	-0.576111	-0.883446	-0.411495	-0.945751	-0.126124

After standardization

Area	Perimeter	Length	Width	Eccentricity	Irregularity
182	57	16.000000	13.000000	0.582961	4.225121
211	64	16.552945	15.231546	0.391516	4.405942
237	67	18.027756	14.866069	0.565685	4.352118
227	66	17.000000	16.000000	0.337915	4.380574
204	60	15.000000	15.000000	0.000000	4.200840
217	66	17.204651	15.000000	0.489760	4.480372
239	66	18.027756	16.155494	0.443760	4.269185
239	66	17.691806	15.556349	0.476274	4.269185
263	69	20.615528	16.401219	0.605854	4.254722
263	72	18.027756	17.088007	0.318651	4.439710

After standardization, every feature has mean = 0 and standard deviation = 1, making features directly comparable.

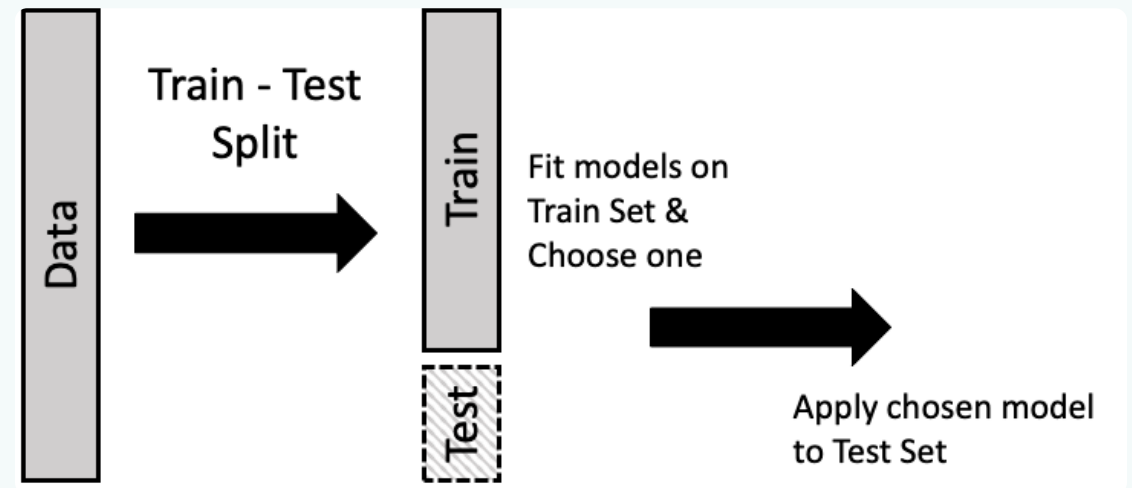
Preparing data: train/test split

To evaluate whether a model **generalizes**, split the **labeled data** into:

- **Training set** — used to fit the model
- **Test set** — held out for final evaluation

A typical split ratio is:

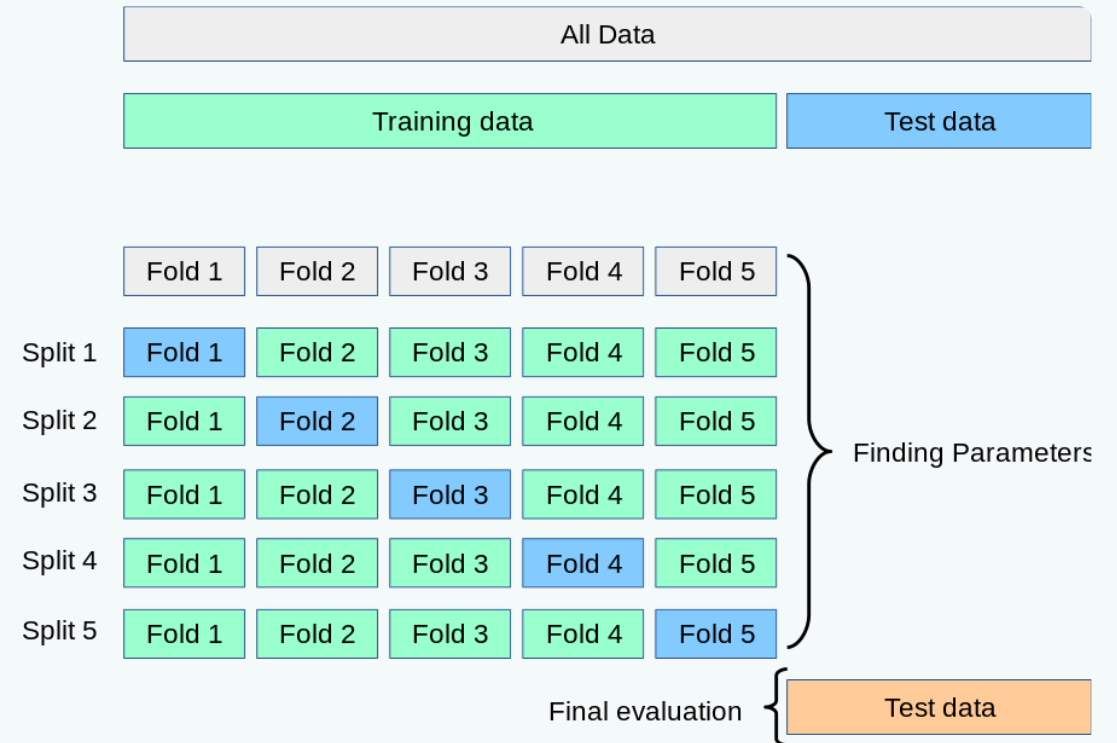
80 % training / 20 % test



Preparing data: cross-validation

A more robust way to select **model parameters**:

1. Split the **training set** into several validation rounds
2. **Train** on one subset, **validate** on another
3. Repeat across folds or random resamples and **average** the results
4. Pick the **parameters** with the best average score
5. Report final **performance** on the held-out **test set**



Evaluating models: accuracy

For classification, the simplest metric is **accuracy**:

$$\text{accuracy} = \text{correct predictions} / \text{total predictions}$$

Accuracy alone can be misleading — a model may achieve high accuracy simply by always predicting the most frequent class.

Always inspect per-class performance, especially with imbalanced datasets.

Evaluating models: confusion matrix

A **confusion matrix** shows the full breakdown of predictions per class:

- **True positives (TP)** — correctly predicted positives
- **True negatives (TN)** — correctly predicted negatives
- **False positives (FP)** — negatives predicted as positive
- **False negatives (FN)** — positives predicted as negative

		Predicted	
		Positive	Negative
Actual	Positive	True positive	False negative
	Negative	False positive	True negative



Evaluating models: precision and recall

Two complementary metrics derived from the **confusion matrix**:

$$\text{precision} = \text{TP} / (\text{TP} + \text{FP})$$

Of all samples predicted as positive, how many truly are?

$$\text{recall} = \text{TP} / (\text{TP} + \text{FN})$$

Of all true positives, how many did the model find?

		Predicted	
		Positive	Negative
Actual	Positive	True positive	False negative
	Negative	False positive	True negative

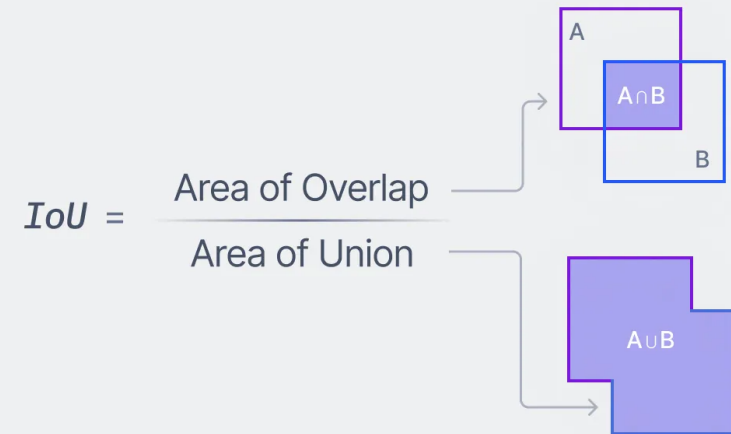


Evaluating segmentation: Intersection over Union

For segmentation tasks, **IoU** (also called the Jaccard index) is the standard metric:

$$\text{IoU} = \text{intersection} / \text{union}$$

It measures the **overlap** between the **predicted mask** and the **ground truth** annotation. A perfect prediction gives **IoU = 1**.



v7

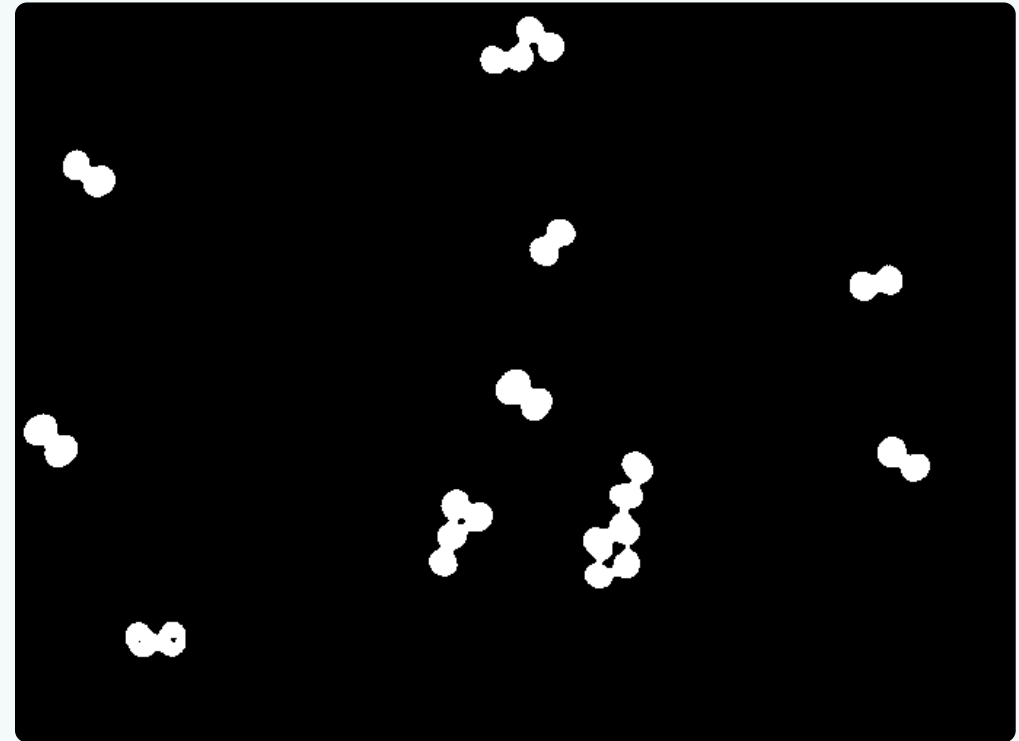
Data annotations: teaching the model

Supervised learning needs **examples with labels**.

For images, annotations can define:

- **pixels** that belong to a class
- **objects** that should be measured
- **ground truth** masks for evaluation

In Notebook 1, we will create pixel labels manually in **napari**.



The scikit-learn mental model

Data

- **X** — features used by the model
- **y** — labels or targets to learn
- rows are **samples**
- columns are **features**

For images, samples can be **pixels** or **objects**.

Methods

```
model.fit(X_train, y_train)
```

learn patterns from the training data

```
model.predict(X_test)
```

apply the trained model to new samples

```
scaler.transform(X)
```

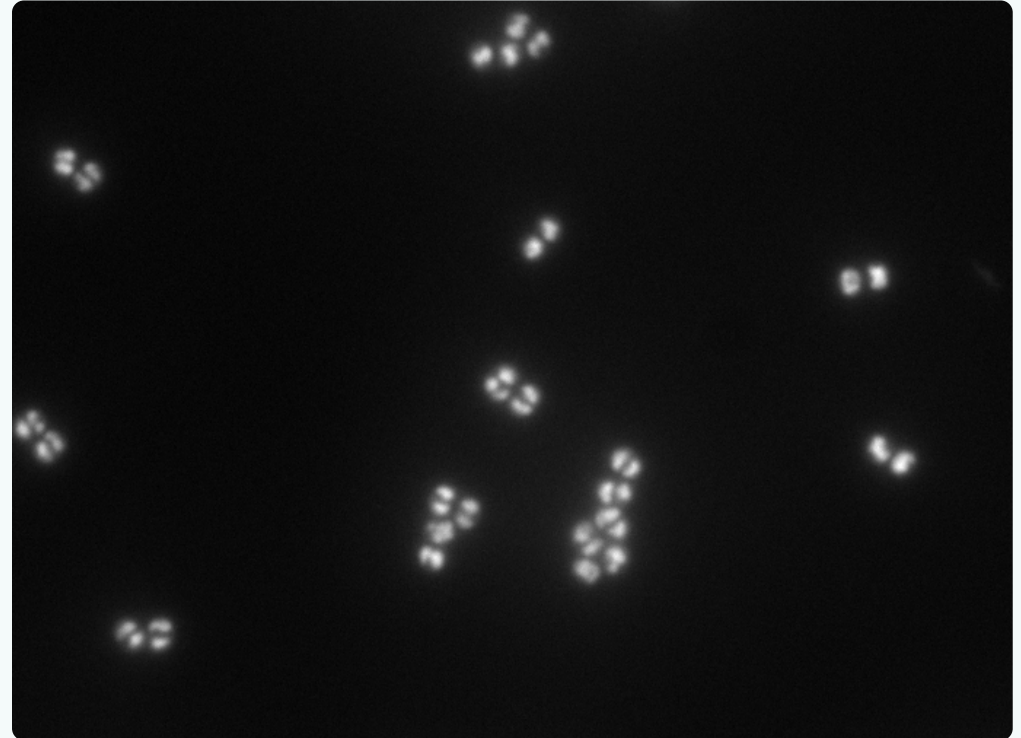
apply a learned preprocessing step

Practical session — Notebook 1

Image segmentation

Applying classical ML to microscopy images of *S. aureus*:

- **Load and normalize** fluorescence images
- **Segment** with **k-means clustering**
- **Evaluate masks** with **Intersection over Union**
- Create **training labels** in **napari**
- Build **feature stacks** with image filters
- Train **Random Forest** and **K-nearest neighbors** segmenters

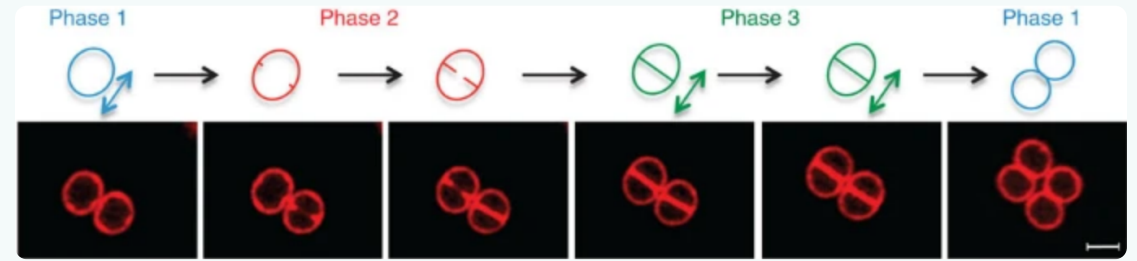


Practical session — Notebook 2

Cell cycle classification

Classifying *S. aureus* cells by cell cycle phase:

- Load and explore data with **Pandas**
- **Shuffle**, **standardize**, and **split** the dataset
- Train and evaluate **ML models**
- Inspect **accuracy**, **confusion matrix**, and **precision**
- Improve **feature choices** and cross-validate with **ShuffleSplit**
- Train and fine-tune an **SVC** with **GridSearchCV**



Practical session — Notebook 3 (*optional*)

Segmentation filtering

Using ML to remove poorly segmented cells:

- **Instance segmentation** from a binary image
- Load manual **good/bad segmentation labels**
- Measure new objects with `regionprops_table`
- **Predict and remove** bad segmentations

	Cell ID	Area	Perimeter	Length	Width	Eccentricity	Well Segmented
0	1	209	71	18.601075	15.000000	0.591364	1
1	2	224	79	19.416488	15.811388	0.580405	1
2	3	263	87	24.166092	15.652476	0.761892	1
3	4	204	76	25.455844	12.041595	0.881042	1
4	5	229	81	20.615528	15.524175	0.657983	1

Getting started

All notebooks, datasets, and images are in the course repository:

<https://github.com/Euro-BioImaging/evolve-python-hybrid-course>

We will work in **JupyterLab**.

Instructions for today are found in the README inside the Day 3 folder